

实验 4 Linux 进程的管道通信

1. 实验目的

了解管道通信的特点，熟悉管道通信的概念和原理，掌握管道通信的实现方法。

2. 相关知识与系统调用

(1) 管道的概念

管道是用于连接一个读进程和一个写进程的 `pipe` 文件。逻辑上是管道文件；物理上是由文件系统的高速缓冲区构成。管道按 FIFO 方式单向传送消息，只允许在建立者及其子进程间使用。

写进程：以字符流的形式将大量数据送入管道，不断从 `pipe` 一端写入数据流，在规定的最大长度（如 4096 字节）范围内，每次写入的信息长度是可变的。

读进程：在需要时从 `pipe` 的另一端读出数据，读出单位长度也是可变的。

(2) pipe 的建立

`int fd[2] ..... pipe(fd)` 在系统打开文件表中建立该 `pipe` 的两个表目

`fd[1]`: `pipe` 入口，控制写操作

`fd[0]`: `pipe` 的出口，控制读操作

(3) pipe 的读写

写操作：`write(fd[1],buf,size)`

把 `buf` 中的长度为 `size` 字符的消息送入管道入口 `fd[1]`

`buf`: 存放消息的空间，`size`: 要写入的字符长度

读操作：`read(fd[0],buf,size)`, 从 `pipe` 出口 `fd[0]` 读出 `size` 字符的消息置入 `buf`

互斥：一个进程正在对 `pipe` 进行读/写时，另一进程必须等待。

同步：

发送进程：写 `pipe` 时，若 `pipe` 文件长度已经到 4096 字节，仍有部分信息没有写入，则进程进入睡眠状态；当读进程收走了全部信息时，被唤醒，将余下信息送入 `pipe` 中。

接收进程：读 `pipe` 时，若 `pipe` 为空，则进入等待状态。一旦有发送进程执行写操作，被唤醒。

(4) 其他相关系统调用

`sleep(n)`: `n` 为整数，为秒(s)级。进程调用后将自动阻塞 `n` 秒，`n` 秒后自动唤醒，进入就绪状态。

用来锁定文件的某些段或者整个文件，本函数使用的头文件为：“`unistd.h`”。

`int lockf(int files, int function, long size)` 其中：`files` 是文件描述符，`function` 是锁定和解锁；1 表示锁定，0 表示解锁。`size` 是锁定和解锁的字节数，若用 0，表示从文件的当前位置到文件尾。

3. 实验程序

【程序 3_11】用 C 语言编写一个程序，建立一个 `pipe`。父进程生成一个子进程，子进程向 `pipe` 中写入一字符串，父进程从 `pipe` 中读出该字符串。如图 3.1 所示。

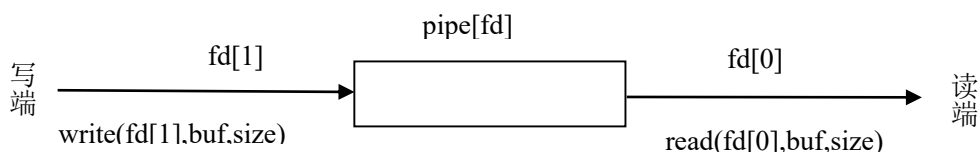


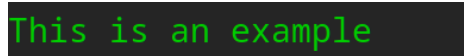
图 3.1 管道通信

```

1. #include <stdio.h>
2. #include <unistd.h>
3. #include <stdlib.h>
4. #include <sys/wait.h>
5. void main(){
6.     int x,fd[2];
7.     char buf[30],s[30];
8.     pipe(fd);           //创建管道
9.     while ((x = fork()) == -1); //创建管道失败时，一直循环
10.    if(x == 0){
11.        sprintf(buf,"This is an example \n");
12.        write(fd[1],buf,30); //将 buf 中字符写入管道
13.    }
14.    else{                //父进程返回
15.        wait(0);
16.        read(fd[0],s,30); //父进程读管道中字符
17.        printf("%s",s);
18.    }
19. }

```

【程序运行结果截图】



【程序 3_12】编写一程序，建立一个管道。同时，父进程生成子进程 p1，p2，这两个子进程分别向管道中写入各自的字符串，父进程读出它们。如图 3.2 所示。

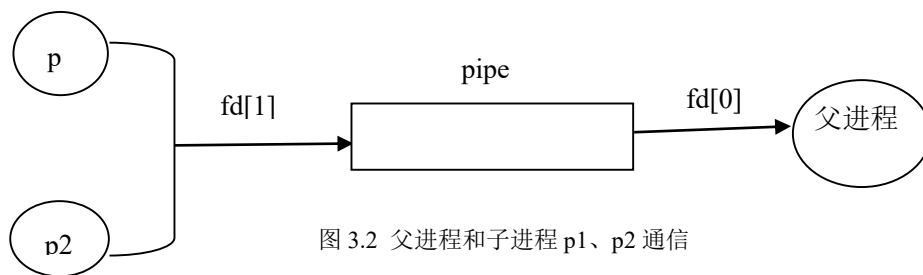


图 3.2 父进程和子进程 p1、p2 通信

```

1. #include <stdio.h>
2. #include <unistd.h>
3. #include <stdlib.h>
4. #include <sys/wait.h>
5. void main(){
6.     int i,r,p1,p2,fd[2];
7.     char buf[50],s[50];
8.     pipe(fd);           //父进程建立管道
9.     while((p1 = fork()) == -1);
10.    if(p1 == 0){

```

```

11.         lockf(fd[1],1,0);           //锁定写入端
12.         sprintf(buf,"messeages from child process p1!\n");
13.         printf("child process p1!\n");
14.         write(fd[1],buf,50);         //将 buf 中的 50 个字符写入管道
15.         sleep(5);                   //睡眠 5 秒，等待父进程读管道
16.         lockf(fd[1],0,0);           //释放管道写入端
17.         exit(0);
18.     }
19.     else{
20.         while((p2 = fork()) == -1);
21.         if(p2 == 0){
22.             lockf(fd[1],1,0);
23.             sprintf(buf,"messeages from child process p2!\n");
24.             printf("child process p2!\n");
25.             write(fd[1],buf,50);      //将 buf 中的 50 个字符写入管道
26.             sleep(5);                 //睡眠 5 秒，等待父进程读管道
27.             lockf(fd[1],0,0);         //释放管道写入端
28.             exit(0);
29.         }
30.         wait(0);
31.         if((r = read(fd[0],s,50)) == -1)
32.             printf("can't read pipe\n");
33.         else
34.             printf("%s",s);
35.         wait(0);
36.         if((r = read(fd[0],s,50)) == -1)
37.             printf("can't read pipe\n");
38.         else
39.             printf("%s",s);
40.         exit(0);
41.     }
42. }

```

【程序运行结果截图】

```

child process p1!
child process p2!
messeages from child process p1!
messeages from child process p2!

```

```

child process p1!
messeages from child process p1!
child process p2!
messeages from child process p2!

```

4. 实验思考与扩充

- (1) 将【程序 3_11】中的父进程作为 pipe 的写端，而子进程作为 pipe 的读端，是否可以？
- (2) 【程序 3_12】的四行输出的顺序是否一定如上面截图所示？为什么？
- (3) 【程序 3_12】中的 sleep(5)起什么作用？
- (4) 【程序 3_12】中若子进程有 3 个，情况会怎样？lockf 这个系统调用的作用是什么？
- (5) 【程序 3_12】中父进程的两段读管道操作代码完全一样，试说明其含义。